**Shubham Kulkarni**
https://www.instagram.com/shubhamkulkarni_insta/

# 📘 Scenario-Based Frontend Interview Q&A Sheet (Part 2) (Find Part 1 questions at the end after these questions)

---

## Q1: How will you handle complex forms with 20+ fields?

**Answer:** When dealing with complex forms containing 20+ fields, scalability, maintainability, and performance are the top priorities. A few strategies I would use:

1. **Form State Management:** Instead of keeping everything in component-level state, I would use libraries like **React Hook Form** or **Formik** because they provide:
   - Built-in validation
   - Better performance (minimizes unnecessary re-renders)
   - Easy integration with UI components
2. **Divide into Logical Sections (Step Forms):** Break down the form into smaller steps/tabs. For example:
   - Step 1: Personal details
   - Step 2: Address details
   - Step 3: Preferences This improves usability and reduces cognitive load.
3. **Dynamic Form Rendering:** Store form configuration (labels, input types, validation rules) in a JSON schema and render inputs dynamically.
   - Example: instead of hardcoding 20+ inputs, loop through a config array.
   - Makes it easy to maintain and update forms in the future.
4. **Validation Strategy:**
   - Client-side validation using Yup/Zod schemas.
   - Server-side validation to avoid bypass.
   - Show validation errors inline to improve UX.

5. **Performance Optimization:**

- Use **debouncing** for expensive validations.
- Lazy load non-essential sections.
- Memoize form field components to prevent unnecessary re-renders.

6. **Accessibility (a11y):**

- Proper labeling with `aria-labels` and `for` attributes.
- Keyboard navigation.

👉 Example:

```
<FormProvider {...methods}>
  <form onSubmit={handleSubmit(onSubmit)}>
    {formSchema.map(field => (
      <Controller
        key={field.name}
        name={field.name}
        control={control}
        render={({ field }) => <TextField {...field} label={field.label} />}
      />
    ))}
  </form>
</FormProvider>
```



---

## Q2: How will you support dark mode in your app?

**Answer:** Dark mode support is both a **UX feature** and a **technical challenge**.

1. **CSS Variables / Custom Properties:**

- Define theme variables ( `--bg-color` , `--text-color` ) and switch between them.
- Avoid hardcoding colors.

```
:root {
  --bg-color: #ffffff;
  --text-color: #000000;
}
[data-theme="dark"] {
```

```
    --bg-color: #121212;
    --text-color: #ffffff;
  }
```

2. **Detect System Preferences:**
   - Use `prefers-color-scheme: dark` media query to auto-detect system theme.
3. **Persistent User Choice:**
   - Store user preference in **localStorage** or database.
   - Example: `localStorage.setItem("theme", "dark")`.
4. **Theming Libraries:**
   - In React, use libraries like **styled-components** with `ThemeProvider` or Tailwind's dark mode support.
5. **Images & Assets:**
   - Provide alternative assets (light/dark logos).
   - Use SVGs that adapt with CSS.
6. **Accessibility Considerations:**
   - Maintain color contrast (minimum WCAG AA standard).
   - Avoid pure black for dark mode (use #121212 for better readability).

👉 Real-world approach:

- On app load → check system preference & stored preference → apply theme → re-render UI.

---

## Q3: How will you secure your app from XSS attacks?

**Answer:** Cross-Site Scripting (XSS) occurs when malicious scripts are injected into a trusted application. To prevent it:

1. **Escape & Sanitize User Input:**
   - Never trust user input.
   - Use libraries like DOMPurify in React for sanitizing HTML.

```
const safeHTML = DOMPurify.sanitize(userInput);
```

2. **Content Security Policy (CSP):**
   - Configure HTTP headers to block inline scripts.
   - Example:

```
Content-Security-Policy: default-src 'self'; script-src 'self'
```

3. **Avoid `dangerouslySetInnerHTML` in React:**
   - Use only when absolutely necessary, and sanitize content first.
4. **Input Validation on Backend:**
   - Even if frontend sanitizes, always validate and escape on the server.
5. **Use Framework Security Defaults:**
   - React auto-escapes data binding ( `<div>{userInput}</div>` ).
   - Avoid overriding these protections.
6. **HttpOnly & Secure Cookies:**
   - Store tokens in HttpOnly cookies instead of localStorage.

👉 Example Attack Prevention:

- If user inputs `<script>alert(1)</script>` , React escapes it and renders as text instead of executing.

---

## Q4: How do you handle online or offline working of your app?

**Answer:** Building offline-first apps improves reliability.

1. **Service Workers (PWA):**
   - Cache static assets & API responses.
   - Use Workbox to manage caching strategies.
2. **Local Storage / IndexedDB for Offline Data:**
   - Save user actions offline and sync later when online.

- Example: A note-taking app saves notes in IndexedDB, then syncs with the server once online.

3. **Detect Online/Offline State:**

- Use `navigator.onLine` or `window.addEventListener("online/offline")`.
- Show UI indicators like "You're offline" banners.

4. **Data Sync Strategies:**

- Queue requests offline and replay them when online.
- Conflict resolution strategies if same data updated in multiple devices.

5. **Graceful Degradation:**

- If offline, show cached data instead of error screens.

👉 Example in React:

```
useEffect(() => {
  const handleOnline = () => setIsOnline(true);
  const handleOffline = () => setIsOnline(false);

  window.addEventListener("online", handleOnline);
  window.addEventListener("offline", handleOffline);

  return () => {
    window.removeEventListener("online", handleOnline);
    window.removeEventListener("offline", handleOffline);
  };
}, []);
```



---

## Q5: How will you solve caching issues in production?

**Answer:** Caching improves performance but can cause **stale data issues**. Solutions:

1. **Cache Busting (Versioning):**

- Add hash to filenames (`main.abc123.js`).
- Ensures browser fetches latest file after deployment.

2. **HTTP Cache Headers:**

- `Cache-Control: no-cache, must-revalidate` for dynamic content.

- `Cache-Control: public, max-age=31536000` for static assets.

3. **Service Worker Updates (PWA):**
   - Update service worker when new build is deployed.
   - Prompt user to refresh for latest version.
4. **Invalidate CDN Cache:**
   - Use CDN APIs to purge outdated files.
   - Example: `aws cloudfront create-invalidation`.
5. **ETags & Last-Modified Headers:**
   - Browser verifies with server if resource changed before using cache.
6. **Real-world Workflow:**
   - Static files → hashed filenames → long cache.
   - API responses → short cache / revalidation.
   - User-specific data → no cache.

👉 Example:

```
Cache-Control: max-age=0, no-cache, no-store, must-revalidate
ETag: "v2.3.1"
```



---

# Other Scenario based interview questions

---

# 1) App slowed down after new features — find & fix

**Symptoms**

- Slow typing, scroll jank, sluggish route changes, long TTI/LCP, high CPU.

**Likely Root Causes**

- Unnecessary re-renders, huge lists, expensive calculations in render, heavy dependencies loaded upfront, chatty state updates, deep prop drilling, stale memoization.

**Step-by-Step Diagnosis**

- **Performance Panel (Chrome DevTools):** Record → look for long tasks (>50ms), scripting spikes, layout thrash.
- **React DevTools Profiler:** Identify "hot" components (render time & render count).
- **Why did this render?** (React DevTools) to trace prop/state changes.
- **Coverage Tab:** Unused JS/CSS bloat.
- **Network Tab:** Large bundles, waterfalls, N+1 API calls.

**Fix Patterns**

- **Stop re-renders:**
  - `React.memo` for pure components.
  - `useCallback` for stable handlers; `useMemo` for heavy computations.
  - Derive less state; avoid setting state in loops/effects.
- **List virtualization:** `react-window` / `react-virtualized`.
- **Code splitting:** `React.lazy` + `Suspense`; dynamic imports for heavy libs (charts, editors).
- **State hygiene:** Keep local state local; avoid global store re-rendering entire trees.
- **Avoid inline objects/arrays:** Hoist constants or memoize.
- **Defer non-critical work:** `requestIdleCallback`, lazy images, `loading="lazy"`.

**Snippets**

```
// Memoized child to cut re-renders
const Row = React.memo(({item, onSelect}) => {/* ... */});

// Stable handler + expensive compute
const onSelect = useCallback((id) => setSel(id), []);
const filtered = useMemo(() => heavyFilter(items, q), [items, q]);

// Virtual list
import { FixedSizeList as List } from 'react-window';
<List height={600} itemCount={rows.length} itemSize={48} width="100%">
```

```
    {({index, style}) => <Row style={style} item={rows[index]} />}
  </List>
```

**Metrics to Watch**

- React Profiler commit time, **LCP** (<2.5s), **CLS** (<0.1), **INP** (<200ms), **JS bundle** size.

**Interview Tip**: Walk through *measure → identify → act → verify* with Profiler screenshots and a before/after metric (e.g., "LCP from 5.3s → 2.2s").

---

# 2) Block login page after auth (guarded & redirect logic)

**Goal**

- Prevent authenticated users from hitting `/login`; protect private routes.

**Approach**

- Central auth state (Context/Redux/Query), route guards, and redirects.

**Snippet**

```
// Auth context
const AuthContext = React.createContext({ user: null });

function PrivateRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <Spinner/>;
  return user ? children : <Navigate to="/login" replace />;
}

function PublicOnlyRoute({ children }) {
  const { user, loading } = useAuth();
  if (loading) return <Spinner/>;
  return user ? <Navigate to="/dashboard" replace /> : children;
}

// Usage (React Router v6)
<Routes>
```

```
  <Route path="/login" element={<PublicOnlyRoute><Login/></PublicOnlyRoute>} />
  <Route path="/dashboard" element={<PrivateRoute><Dashboard/></PrivateRoute>} />
</Routes>
```

**Extras**

- Persist session (httpOnly cookies / secure storage), refresh-token rotation, 401 interceptor to logout.

**Pitfalls**

- Race conditions on page refresh → add “auth bootstrap” loader.
- Flash of `/login` → gate rendering until auth resolved.

---

# 3) Graceful API error handling & resilient UX

**Principles**

- Detect & categorize (network/4xx/5xx/timeout), show actionable UI, offer retry, don’t dead-end.

**Patterns**

- **Data layer**: React Query/SWR with retry/backoff, stale-while-revalidate.
- **UI**: toasts for transient errors, inline messages for blocking errors, retry button.
- **Auth**: on 401 → refresh token or redirect.

**Snippet (React Query)**

```
const { data, error, isError, isLoading, refetch } = useQuery({
  queryKey: ['feed'],
  queryFn: fetchFeed,
  retry: 2, retryDelay: attempt => Math.min(1000 * 2 ** attempt, 8000)
});

if (isLoading) return <SkeletonFeed/>;
if (isError) return <ErrorBlock msg={mapErr(error)} onRetry={refetch}/>;
return <Feed items={data}/>;
```

**Error Boundary**

```
function ErrorBoundary({children}) {
  return (
    <React.Suspense fallback={<Skeleton/>}>
      {children}
    </React.Suspense>
  );
}
```

**Interview Tip**: Mention *observability*: capture errors to Sentry, log correlation IDs, and expose *user-safe* messages.

---

# 4) Duplicate fetching — eliminate overfetch & N+1s

### Symptoms

- Same endpoint called by multiple siblings, repeated on tab switches, rapid re-renders trigger refetch.

### Solutions

- **Single source of truth**: React Query/SWR caches; colocation with shared keys.
- **Lift & share**: Fetch in parent/provider; pass data down.
- **Batching**: Combine endpoints server-side; debounce search; cancel stale requests (AbortController).

### Snippet

```
// Shared cache
const { data: profile } = useQuery({ queryKey: ['profile'], queryFn: getProfile, staleTime:

// Abort stale
const ctrl = new AbortController();
fetch(url, { signal: ctrl.signal });
// on new effect: ctrl.abort()
```

### Pitfalls

- Missing `queryKey` dependencies; re-creating clients/providers per render.

---

# 5) Bundle size optimization & loading strategy

### Diagnose

- **Coverage** (unused JS/CSS), **Network** waterfalls, **Source Map Explorer**/Webpack Analyzer.

### Tactics

- **Split by route & feature**: `React.lazy`, dynamic imports.
- **Tree-shakeable libs**: Prefer ESM, named imports. Replace heavy libs (moment → date-fns/dayjs; lodash → per-method).
- **Defer non-critical**: `import()` in event handlers, prefetch/Preload, lazy images ( `loading="lazy"` ).
- **Compress & cache**: Gzip/Brotli, HTTP caching, immutable hashing.

### Snippet

```
const Chart = React.lazy(() => import('./Chart'));
<Route path="/analytics" element={
  <Suspense fallback={<Spinner/>}><Chart/></Suspense>
}/>
```

### Targets

- Main bundle < 200–300KB gz, fast 3G TTI < 5s.

---

# 6) Handling big forms (20+ fields, multi-step)

### Goals

- Minimal re-renders, schema validation, accessible errors, autosave, undo.

### Tooling

- **React Hook Form** (perf via uncontrolled inputs), **Yup/Zod** for schemas.

### Snippet

```
const schema = z.object({
  email: z.string().email(),
  age: z.number().min(18),
});
const { register, handleSubmit, formState:{errors, isSubmitting} } = useForm({
  resolver: zodResolver(schema), mode: 'onBlur'
});
return (
  <form onSubmit={handleSubmit(onSubmit)}>
    <input {...register('email')} />
    {errors.email && <span>{errors.email.message}</span>}
    <button disabled={isSubmitting}>Submit</button>
  </form>
);
```

**Patterns**

- **useFieldArray** for dynamic lists, **useReducer** for wizard state, **debounced autosave**, optimistic UI.

**Pitfalls**

- Controlled inputs everywhere → re-render storms; missing `aria-*` → poor accessibility.

---

# 7) Dark mode / theming at scale

**Approach**

- **CSS variables** at `:root`, theme class on `<html>` (`.dark`), persisted preference + system fallback.

**Snippet**

```
:root { --bg: #fff; --fg: #111; }
.dark { --bg: #0b0b0b; --fg: #f3f3f3; }
body { background: var(--bg); color: var(--fg); }
```

```
const ThemeContext = React.createContext();
function ThemeProvider({children}) {
  const [theme, setTheme] = useState(getInitialTheme()); // 'light' | 'dark' | 'system'
  useEffect(() => {
    const isDark = theme === 'dark' || (theme==='system' && matchMedia('(prefers-color-schem
    document.documentElement.classList.toggle('dark', isDark);
    localStorage.setItem('theme', theme);
  }, [theme]);
  return <ThemeContext.Provider value={{theme,setTheme}}>{children}</ThemeContext.Provider>;
}
```

**Extras**

- Themed assets (logos), SSR hydration (avoid flash), media-query listener for live system changes.

---

# 8) Prevent XSS & unsafe HTML

**Threats**

- Rendering untrusted HTML, query-param reflection, stored XSS via comments/chat.

**Defenses**

- **Never** render raw HTML; if required, **sanitize** with DOMPurify.
- Escape dynamic content; use template literals safely.
- Avoid string-building URLs; use `URLSearchParams`.
- Set strict CSP on server; prefer httpOnly cookies for tokens.

**Snippet**

```
import DOMPurify from 'dompurify';

function SafeHTML({ html }) {
  const clean = useMemo(() => DOMPurify.sanitize(html), [html]);
  return <div dangerouslySetInnerHTML={{ __html: clean }} />;
}
```

### Interview Angle

- Mention **CSP**, **Trusted Types** (where supported), and **auto-escaping** in frameworks.

---

# 9) Global loading UX & skeletons

### Goals

- Communicate state without blocking; avoid spinner-only screens; handle parallel requests gracefully.

### Patterns

- **Per-query status** with React Query; **global counter** for app-wide loading bar (NProgress).
- **Skeletons** for content blocks; **progressive hydration/streaming** on SSR.

### Snippet

```
// NProgress with fetch counter
const LoadingContext = React.createContext({ count: 0, inc: ()=>{}, dec: ()=>{} });

// Skeleton usage
{isLoading ? <CardSkeleton/> : <Card data={data}/>}
```

### Pitfalls

- Infinite spinner when a request fails; no retry. Ensure error → actionable message + retry.

---

# 10) Make it work on slow networks & low-end devices

### Performance Budget

- <1MB total (gzipped) on first load, minimal main-thread work, images optimized.

### Tactics

- **Images**: responsive `srcset`, `sizes`, WebP/AVIF, lazy load, blur-up placeholders.
- **JS**: split by route/feature, avoid heavy polyfills, use `module/nomodule` fallback if needed.

- **CSS**: critical CSS inline, rest deferred; avoid huge frameworks if unused.
- **Caching**: service worker (Workbox) for offline & pre-cache shell; HTTP caching with immutable hashes.
- **Data**: prefetch on hover, cache with SWR/Query; compress JSON (GZIP/Brotli).

### Snippets

```
<img src="img-800.webp"
     srcset="img-400.webp 400w, img-800.webp 800w, img-1200.webp 1200w"
     sizes="(max-width: 600px) 400px, 800px"
     loading="lazy" alt="...">
```

```
// Workbox basic pre-cache
workbox.precaching.precacheAndRoute(self.__WB_MANIFEST);
```

### Validation

- Lighthouse/Pagespeed CI; track **LCP/CLS/INP**, JS bytes, image bytes, CPU time on mid-tier device.

---